MINISTRY OF EDUCATION, CULTURE, RESEARCH, AND TECHNOLOGY
MALANG STATE POLYTECHNIC
DEPARTMENT OF INFORMATION TECHNOLOGY
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Phone. (0341) 404424 – 404425, Fax (0341) 404420
http://www.polinema.ac.id

Courses	:	Advanced Web Programming (PWL)
Courses	:	D4 – Informatics Engineering / D4 – Business Information Systems
Semester	:	4 (four) / 5 (five)
The meeting	:	6 (six)

JOBSHEET 06

Ajax Form (AdminLTE) and Client Validation

The process of creating a CRUD (Create, Read, Update, Delete) form with validation in Laravel 10 using jQuery Validation involves several important steps that include database setup, model creation and migration, controller development, view writing, and adding form validation on the client side. *Client side from validation* is done more on the browser side and not for security purposes, but more for user convenience. Meanwhile, *server-side validation* is carried out on the server side for security purposes by *filtering* all incoming *requests* before finally being processed further.

One popular way to perform client-side validation is to use the jQuery Validation plugin. The ***jQuery Validation*** plugin is used to add client-side validation to forms. For example, we can set an input to be mandatory and not more than 255 characters. This validation helps in providing direct feedback to the user about input errors without the need to reload the page or send *a request* to the server.

In accordance with `Case Study PWL.pdf`. So our Laravel 10 project is still the same using the **PWL_POS repository.**

We will use **Project PWL_POS** until the 12th meeting, as the project we will learn

A. *AJAX form*

AJAX (Asynchronous JavaScript and XML) is a technique or method in web development that allows web applications to send and receive data from the server asynchronously (without reloading the entire page). With AJAX, the interaction between the client and the server becomes more dynamic and responsive, as users can interact with the web page and see the changes directly without having to refresh the page.

Ajax form is a technique in which an HTML form is sent to a server asynchronously using AJAX, without reloading the entire web page. With AJAX forms, you can send data to the server and dynamically display the response on the page, thereby improving the user experience by making interactions faster and more responsive.

Why Use AJAX Form?

1. **Instant Response**: AJAX allows you to send data and receive responses from the server without the need to reload the page.
2. **Better User Experience**: Since there is no page reload, the app feels faster and more interactive, similar to the desktop app.

Reduced Server Load: By sending only the data that is needed, AJAX can reduce bandwidth usage and load on the server.

B. Client-Side Validation

Client-side validation is the process of checking the data entered by the user on a web form before it is sent to the server. This validation is done using code that runs in the user's browser, such as JavaScript, and aims to ensure that the data entered complies with certain rules, such as the correct email format, the appropriate character length, or the absence of blank fields that must be filled in.

Objectives and Benefits of Validation on the Client Side

1. **Instant Feedback**
   Users get feedback as soon as they enter invalid data, such as email formatting errors or unfilled fields. This improves the user experience because they don't have to wait for a response from the server to know if their input is correct or incorrect.
2. **Reducing Server Load**
   By performing validation on the client side, errors can be identified and corrected before the data is sent to the server, so the server does not have to process invalid requests. This can reduce the server workload and improve the performance of the application.
3. **Increase Efficiency**
   Client-side validation helps prevent invalid data submissions, thereby reducing the number of HTTP requests that need to be processed by the server. This saves bandwidth and processing time, making the application more efficient.
4. **Ensuring Data Integrity**

With client-side validation, many input errors can be prevented before the data reaches the server. For example, make sure that the phone number contains only numbers or email addresses following the correct format.

5. **Simplify the Development Process**
   With validation on the client side, developers can handle many potential input errors upfront, which simplifies the processing logic on the server side. This allows developers to focus on more complex validations or other business logic on the server.
6. **Improving Security**
   While client-side validation cannot replace server-side validation (as it can be easily ignored or manipulated by experienced users), it can still be helpful in reducing the amount of invalid data reaching the server. It serves as the first layer of defense, preventing some types of unwanted inputs.
7. **Provide User Guide**
   Client-side validation allows developers to provide better guidance and instructions to users on how to enter data correctly. For example, an error message could be displayed under the wrong column, giving the user specific instructions

How Does Client-Side Validation Work?

Client-side validation is usually done using JavaScript or JavaScript frameworks such as jQuery. Here's a simple example of form validation on the client side:

```
<! DOCTYPE html>
<html>
<head>
    <title>Client-Side Validation Example</title>
</head>
<body>
    <form name="myForm" onsubmit="return validateForm()" method="post">
 Name: <input type="text" name="name"><br><br>
 Email: <input type="text" name="email"><br><br>
        <input type="submit" value="Submit">
    </form>
    <script>
        function validateForm() {
            var email = document.forms["myForm"]["email"].value;
            var name = document.forms["myForm"]["name"].value;

            if (name == "") {
 alert("Name must be filled out");
                return false;
}

            var emailPattern = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[ a-zA-Z]{2,6}$/;
            if (!emailPattern.test(email)) {
 alert("Please enter a valid email address");
                return false;
}
            return true;
}
```

```
    </script>
</body>
</html>
```

Advantages of Client-Side Validation

1. **Responsive**
   Users get a quick response to their input without having to wait for interaction with the server.
2. **Interactive**
   It can provide additional and more contextual instructions to the user to correct errors.
3. **Resource Savings**
   Reduce the number of unnecessary requests to the server, thus saving server bandwidth and resources.

Validation Limitations on the Client Side

1. **Not Replacing Server-Side Validation**
   Client-side validation can be bypassed by experienced users or automated devices. Therefore, client-side validation should always be complemented by server-side validation to ensure data security and integrity.
2. **Reliance on JavaScript**
   If the user disables JavaScript in their browser, client-side validation will not work.

Client-side validation is a critical component in modern web application development, as it improves user experience and application efficiency. However, this should always be used in conjunction with server-side validation to maintain security and ensure the data received by the application is valid and compliant with business rules.

C. jQuery Validation

One popular way to perform client-side validation is to use the jQuery Validation plugin. **jQuery Validation** is a jQuery plugin that is used to validate HTML forms on the client side efficiently and interactively. The plugin makes it easy for developers to add validation logic to forms in an easy and customizable way, providing users with direct feedback on their input errors before the data is sent to the server.

Key Features of jQuery Validation

1. **Ease of Use**

jQuery Validation is designed to be easy to integrate and use. With a few lines of code, we can add validation to the HTML form without the need to write validation logic from scratch.

2. **Real-Time Validation**
   This plugin validates form input in real-time as the user types or after they move from one field to another. It provides direct feedback to the user regarding their input errors.
3. **Out-of-the-box validation rules**:
   jQuery Validation provides a variety of ready-to-use validation rules, such as:
   - *required*: Ensures that the field is not empty.
   - *email*: Ensure that the input is formatted with a valid email address.
   - *url*: Ensures that the input is in a valid URL format.
   - *minlength* and *maxlength*: Limits the minimum or maximum number of characters in the input.
   - *number*: Ensures that the input contains only numbers.
4. **Custom Error Messages**
   We can customize the error message displayed to the user. For example, you can change the default message like "This field is required" to something more specific or appropriate to the context of your application.
5. **Integration with jQuery UI**
   jQuery Validation can be easily integrated with jQuery UI to display error messages in a more attractive format, such as using tooltips or dialog boxes.
6. **Multi-Field Validation**
   This plugin supports validation involving more than one field. For example, we can make sure that the two password fields and the password confirmation have the same value.
7. **Plugins and Extensions**
   jQuery Validation has an ecosystem of plugins and extensions that allow you to add custom validation rules or change the default behavior.

How to Use jQuery Validation

Here's a simple example of how jQuery Validation is used to validate forms:

```
<! DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>jQuery Validation Example</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.12.1/jquery-
ui.css">
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

```
    <script src="https://cdnjs.cloudflare.com/ajax/libs/jquery-
validate/1.19.3/jquery.validate.min.js"></script>
</head>
<body>
    <form id="myForm">
        <label for="name">Name:</label><input type="text" name="name" id="name"><br>
        <label for="email">Email:</label><input type="text" name="email" id="email"><br>
        <input type="submit" value="Submit">
    </form>

    <script>
 $(document).ready(function() {
 $("#myForm").validate({
rules: {
 name: "required",
email: {
 required: true,
 Email: true
}
},
messages: {
 name: "Please enter your name",
 email: "Please enter a valid email address"
}
});
});
    </script>
</body>
</html>
```

Explanation:

- ***rules***: defines the validation rules for each field. In the example above:
    - *Field name* must be filled in (required).
    - *The email field* must be filled in and must be in a valid email format (email).
- **messages**: defines the error message that will be displayed if the validation rules are not met.

Advantages of Using jQuery Validation

1. **Better User Experience**
   Users get immediate feedback, which helps them fix input errors quickly.
2. **Server Load Reduction**
   Client-side validation reduces the number of invalid requests sent to the server, saving server resources.
3. **Flexibility and Customization**
   The plugin is highly flexible and can be customized as per the needs of the application, from validation rules to displayed error messages.
4. **Compatibility with All Modern Browsers**
   jQuery Validation is compatible with almost all modern browsers, so it can be used in a variety of user environments.

jQuery Validation has a variety of built-in methods that are very useful for validating forms on the client side. In addition to standard methods such as required, email, and number, we can also add custom validation methods using addMethod. This allows us to create more specific validation rules as per the needs of our application.

D. Method jQuery Validation

jQuery Validation provides several built-in methods that can be used to validate forms with different types of rules. In addition, jQuery Validation also allows developers to add custom methods with addMethod, as previously described. Here are some additional methods available in jQuery Validation

It	Method	Description
1	*Required*	• Make sure that the field is not empty. • Example: `required: true`
2	*Email*	• Ensure that the input is formatted with a valid email address. • Example: `email: true`
3	*URL*	• Ensure that the input is in a valid URL format. • Example: `url: true`
4	*Date*	• Ensure that the input is in a valid date format (based on regional settings) • Example: `date: true`
5	*dateISO*	• Ensuring that the input is in a valid date format in ISO FORMAT (YYYY-MM-DD) • Example: `dateISO: true`
6	*number*	• Ensure that the input contains only numbers (integers or decimals). • Example: `number: true`
7	*digits*	• Ensure that the input contains only numbers (no decimals). • Example: `digits: true`
8	*creditcard*	• Ensure that the input is formatted as a valid credit card number. • Example: `creditcard: true`
9	*equalTo*	• Make sure that the value of the form element is the same as the other elements (for example, for password confirmation). • Example: `equalTo: "#password"`
10	*maxlength*	• Ensure that the input does not exceed a certain number of characters. • Example: `maxlength: 10`
11	*minlength*	• Ensure that the input has a minimum number of characters. • Example: `minlength: 5`
12	*rangelength*	• Ensure that the input length is within a certain character range. • Example: `rangelength: [5, 10]`

13	*Range*	• Ensure that the input value is within a certain range (e.g., numbers 1 to 100) • Example: `range: [1, 100]`
14	*Max*	• Ensure that the input value does not exceed a certain maximum number. • Example: `max: 100`
15	*Min*	• Ensure that the input value is not less than a certain minimum number. • Example: `min: 1`
16	*Remote*	• Validating a value by sending a request to the server to check if the value is valid or available (for example, checking username availability) • Example `remote: { url: "/check-username", type: "post" }`
17	*step*	• Ensures that the input value is a multiple of a given number (useful for decimal number validation). • Example: `step: 10`
18	*phoneUS*	• Ensure that the input is formatted as a valid phone number in the US. • Example: `phoneUS: true`
19	*Extension*	• Make sure that the uploaded file has a specific extension. • Example: `extension: "jpg|png|gif"`
20	*Accept*	• Ensure that the uploaded file has a specific MIME type. • Example: `accept: "image/*"`
21	*exactlength*	• Ensure that the input contains only characters that are exactly the same length as the provision. • Example: `exactlength: 10`

jQuery Validation is a very useful tool for ensuring that the data entered into a web form is valid and compliant with the rules set before it is sent to the server. This improves the user experience, reduces errors, and makes it easier to manage form validation on the client side in web application development.

E. Jobsheet Practicum

We immediately practiced using Ajax forms and validation on the client side.

Practicum 1. Ajax Capital Add Data (User Data)

1. We create a form to add new data by applying capital and ajax processes.
2. The first thing we set up is to add ***the jQuery Validation*** and ***Sweetalert libraries*** to our web application. The way we add the links of the two *libraries* to `the template.blade.php`, the libraries have been provided by adminLTE.

template.blade.php

```blade
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>AdminLTE 3 | Blank Page</title>

    <meta name="csrf-token" content="{{ csrf_token() }}">

    <!-- Font Awesome -->
    <link rel="stylesheet" href="{{  asset('plugins/fontawesome-free/css/all.min.css') }}">
    <!-- Datatables -->
    <link rel="stylesheet" href="{{  asset('plugins/datatables-bs4/css/dataTables.bootstrap4.min.css') }}">
    <link rel="stylesheet" href="{{  asset('plugins/datatables-responsive/css/responsive.bootstrap4.min.css') }}">
    <link rel="stylesheet" href="{{  asset('plugins/datatables-buttons/css/buttons.bootstrap4.min.css') }}">
    <!-- SweetAlert2 -->
    <link rel="stylesheet" href="{{ asset('plugins/sweetalert2-theme-bootstrap-4/bootstrap-4.min.css') }}">
    <!-- Theme style -->
    <link rel="stylesheet" href="{{  asset('dist/css/adminlte.min.css') }}">

    @stack('css')
</head>
```

template.blade.php

```blade
    <script src="{{ asset('plugins/pdfmake/pdfmake.min.js') }}"></script>
    <script src="{{ asset('plugins/pdfmake/vfs_fonts.js') }}"></script>
    <script src="{{ asset('plugins/datatables-buttons/js/buttons.html5.min.js') }}"></script>
    <script src="{{ asset('plugins/datatables-buttons/js/buttons.print.min.js') }}"></script>
    <script src="{{ asset('plugins/datatables-buttons/js/buttons.colVis.min.js') }}"></script>

    <!-- jquery-validation -->
    <script src="{{ asset('plugins/jquery-validation/jquery.validate.min.js') }}"></script>
    <script src="{{ asset('plugins/jquery-validation/additional-methods.min.js') }}"></script>

    <!-- SweetAlert2 -->
    <script src="{{ asset('plugins/sweetalert2/sweetalert2.min.js') }}"></script>

    <!-- AdminLTE App -->
    <script src="{{ asset('dist/js/adminlte.min.js') }}"></script>
    <script>
        $.ajaxSetup({
            headers: {
                'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content')
            }
        });
    </script>

    @stack('js')
    </body>
    </html>
```

3. Next, we modify the view `user/index.blade.php`, we add a button to create an ajax popup form

```
@extends('layouts.template')

@section('content')
  <div class="card card-outline card-primary">
    <div class="card-header">
      <h3 class="card-title">{{ $page->title }}</h3>
      <div class="card-tools">
        <a class="btn btn-sm btn-primary mt-1" href="{{ url('user/create') }}">Tambah</a>
        <button onclick="modalAction('{{ url('user/create_ajax') }}')" class="btn btn-sm btn-success mt-1">Tambah Ajax</button>
      </div>
    </div>
    <div class="card-body">
      @if (@session('success'))
          <div class="alert alert-success">{{ session('success')}}</div>
      @endif
      @if (session('error'))
          <div class="alert alert-danger">{{session('error')}}</div>
      @endif
      <div class="row">
          <div class="col-md-12">
              <div class="form-group row">
                  <label class="col-1 control-label col-form-label">Filter:</label>
                  <div class="col-3">
                      <select class="form-control" id="level_id" name="level_id" required>
                          <option value="">- Semua -</option>
                          @foreach ($level as $item)
                              <option value="{{ $item->level_id }}">{{ $item->level_nama }}</option>
                          @endforeach
```

We add the following code, to create a modal form to add user data with ajax

```
<button onclick="modalAction('{{ url('/user/create_ajax') }}')" class="btn btn-sm btn-success mt-1">Add Ajax</button>
```

4. Next we add the following code at the **end of** `@section('content')` in the `user/index.blade.php view`

```
<div id="myModal" class="modal fade animate shake" tabindex="-1" role="dialog" data-backdrop="static" data-keyboard="false" data-width="75%" aria-hidden="true"></div>
```

5. Then we add the following code at the **beginning** `of the @push('js')` in the user`/index.blade.php view`

```
function modalAction(url= ''){
 $('#myModal').load(url,function(){
 $('#myModal').modal('show');
});
}
```

So that the display of the program code will be as follows

```
  <div id="myModal" class="modal fade animate shake" tabindex="-1" role="dialog" data-backdrop="static"
  data-keyboard="false" data-width="75%" aria-hidden="true"></div>
@endsection

@push('css')
@endpush

@push('js')
  <script>
    function modalAction(url = ''){
        $('#myModal').load(url,function(){
            $('#myModal').modal('show');
        });
    }

    var dataUser;
    $(document).ready(function() {
        dataUser = $('#table_user').DataTable({
            // serverSide: true, jika ingin menggunakan server side processing
            serverSide: true,
            ajax: {
                "url": "{{ url('user/list') }}",
                "dataType": "json",
                "type": "POST",
                "data": function (d){
                    d.level_id = $('#level_id').val();
                }
            },
            columns: [ {
```

Change it like this

6. Next, we modify the `route/web.php` to accommodate ajax operations

```
Route::group(['prefix' => 'user'], function () {
    Route::get('/', [UserController::class, 'index']);          // Menampilkan halaman awal user
    Route::post('/list', [UserController::class, 'list']);      // Menampilkan data user dalam bentuk json untuk datatables
    Route::get('/create', [UserController::class, 'create']);   // Menampilkan halaman form tambah user
    Route::post('/', [UserController::class, 'store']);         // Menyimpan data user baru
    Route::get('/create_ajax', [UserController::class, 'create_ajax']);   // Menampilkan halaman form tambah user Ajax
    Route::post('/ajax', [UserController::class, 'store_ajax']);         // Menyimpan data user baru Ajax
    Route::get('/{id}', [UserController::class, 'show']);       // Menampilkan detail user
    Route::get('/{id}/edit', [UserController::class, 'edit']);  // Menampilkan halaman form edit user
    Route::put('/{id}', [UserController::class, 'update']);     // Menyimpan perubahan data user
    Route::delete('/{id}', [UserController::class, 'destroy']); // Menghapus data user
});
```

7. Then we add the `create_ajax() function` to the UserController.php file

```
public function create_ajax()
{
    $level = LevelModel::select('level_id', 'level_nama')->get();

    return  view('user.create_ajax')
                ->with('level', $level);
}
```

8. After that, we create **a new view of** the `user/create_ajax.blade.php` to display the form with ajax

```
<form action="{{ url('/user/ajax') }}" method="POST" id="form-add">
@csrf
<div id="modal-master" class="modal-dialog-modal-lg" role="document">
    <div class="modal-content">
```

```
        <div class="modal-header">
            <h5 class="modal-title" id="exampleModalLabel">Add User</h5 Data>
            <button type="button" class="close" data-dismiss="modal" aria-label="Close"><span
aria-hidden="true">&times;</span></button>
        </div>
        <div class="modal-body">
            <div class="form-group">
                <label>User Level</label>
                <select name="level_id" id="level_id" class="form-control" required>
                    <option value="">- Select Level -</option>
                    @foreach($level as $l)
                        <option value="{{ $l->level_id }}">{{ $l->level_nama }}</option>
                    @endforeach
                </select>
                <small id="error-level_id" class="error-text form-text text-danger"></small>
            </div>
            <div class="form-group">
                <label>Username</label>
                <input value="" type="text" name="username" id="username" class="form-control"
required>
                <small id="error-username" class="error-text form-text text-danger"></small>
            </div>
            <div class="form-group">
                <Label>Name</label>
                <input value="" type="text" name="name" id="name" class="form-control"
required>
                <small id="error-name" class="error-text form-text text-danger"></small>
            </div>
            <div class="form-group">
                <label>Password</label>
                <input value="" type="password" name="password" id="password" class="form-
control" required>
                <small id="error-password" class="error-text form-text text-danger"></small>
            </div>
        </div>
        <div class="modal-footer">
            <button type="button" data-dismiss="modal" class="btn btn-warning">Cancel</button>
            <button type="submit" class="btn btn-primary">Save</button>
        </div>
    </div>
</div>
</form>
<script>
 $(document).ready(function() {
 $("#form-add").validate({
rules: {
 level_id: {required: true, number: true},
 username: {required: true, minlength: 3, maxlength: 20},
 name: {required: true, minlength: 3, maxlength: 100},
 password: {required: true, minlength: 6, maxlength: 20}
},
 submitHandler: function(form) {
$.ajax({
url: form.action,
type: form.method,
data: $(form).serialize(),
 success: function(response) {
                        if(response.status){
 $('#myModal').modal('hide');
Swal.fire({
 icon: 'success',
 title: 'Succeed',
text: response.message
});
dataUser.ajax.reload();
 }else{
```

```
 $('.error-text').text('');
 $.each(response.msgField, function(prefix, val) {
 $('#error-'+prefix).text(val[0]);
});
Swal.fire({
 icon: 'error',
 title: 'Something Went Wrong',
text: response.message
});
}
}
});
                return false;
},
 errorElement: 'span',
 errorPlacement: function (error, element) {
 error.addClass('invalid-feedback');
 element.closest('.form-group').append(error);
},
 highlight: function (element, errorClass, validClass) {
 $(element).addClass('is-invalid');
},
 unhighlight: function (element, errorClass, validClass) {
 $(element).removeClass('is-invalid');
}
});
});
</script>
```

9. Then to accommodate the process of storing data via ajax, we create a `store_ajax()` function on `UserController.php`

```php
public function store_ajax(Request $request) {
    // cek apakah request berupa ajax
    if($request->ajax() || $request->wantsJson()){
        $rules = [
            'level_id' => 'required|integer',
            'username' => 'required|string|min:3|unique:m_user,username',
            'nama'     => 'required|string|max:100',
            'password' => 'required|min:6'
        ];

        // use Illuminate\Support\Facades\Validator;
        $validator = Validator::make($request->all(), $rules);

        if($validator->fails()){
            return response()->json([
                'status' => false, // response status, false: error/gagal, true: berhasil
                'message' => 'Validasi Gagal',
                'msgField' => $validator->errors(), // pesan error validasi
            ]);
        }

        UserModel::create($request->all());
        return response()->json([
            'status' => true,
            'message' => 'Data user berhasil disimpan'
        ]);
    }
    redirect('/');
}
```

10. OK, now let's try to do the process of adding user data using the ajax form. Observe what happened and report it on your *jobsheet* and *commit* reports on github!!

Practicum 2. Ajax Modal Edit Data (User Data)

1. In this Practicum 2, we will do the coding for the editing process using ajax.
2. First of all, let's **change** the list() function on the UserController.php to replace the edit button **to be able to use the**

```php
Fetch user data in json form for datatables
public function list(Request $request)
{
 $users = UserModel::select('user_id', 'username', 'name', 'level_id')
 ->with('level');

    Filter user data by level_id
    if ($request->level_id){
 $users->where('level_id',$request->level_id);
}

    return DataTables::of($users)
 ->addIndexColumn()  adds index/no sort column (default column name: DT_RowIndex)
 ->addColumn('action', function ($user) {  add action column
            /* $btn = '<a href="'.url('/user/' . $user->user_id).'" class="btn btn-info btn-
sm">Detail</a> ';
```

```
$btn .= '<a href="'.url('/user/' . $user->user_id . '/edit').'" class="btn btn-warning btn-
sm">Edit</a> ';
$btn .= '<form class="d-inline-block" method="POST" action="'. url('/user/'.$user->user_id).'"
>'
. csrf_field() . method_field('DELETE') .
'<button type="submit" class="btn btn-danger btn-sm" onclick="return confirm(\'Did you delete
this data?\');" >Delete</button></form>';*/
 $btn = '<button onclick="modalAction(\''.url('/user/' . $user->user_id . '/show_ajax').'\')"
class="btn btn-info btn-sm">Detail</button> ';
 $btn .= '<button onclick="modalAction(\''.url('/user/' . $user->user_id . '/edit_ajax').'\')"
class="btn btn-warning btn-sm">Edit</button> ';
 $btn .= '<button onclick="modalAction(\''.url('/user/' . $user->user_id .
'/delete_ajax').'\')" class="btn btn-danger btn-sm">Delete</button> ';

            return $btn;
})
 ->rawColumns(['action'])  tells you that the action column is html
 ->make(true);
}
```

3. Next, we modify the routes/web.php to accommodate edit requests using ajax

```
Route::group(['prefix' => 'user'], function () {
    Route::get('/', [UserController::class, 'index']);          // Menampilkan halaman awal user
    Route::post('/list', [UserController::class, 'list']);      // Menampilkan data user dalam bentuk json untuk datatables
    Route::get('/create', [UserController::class, 'create']);   // Menampilkan halaman form tambah user
    Route::post('/', [UserController::class, 'store']);         // Menyimpan data user baru
    Route::get('/create_ajax', [UserController::class, 'create_ajax']);   // Menampilkan halaman form tambah user Ajax
    Route::post('/ajax', [UserController::class, 'store_ajax']);         // Menyimpan data user baru Ajax
    Route::get('/{id}', [UserController::class, 'show']);       // Menampilkan detail user
    Route::get('/{id}/edit', [UserController::class, 'edit']);  // Menampilkan halaman form edit user
    Route::put('/{id}', [UserController::class, 'update']);     // Menyimpan perubahan data user
    Route::get('/{id}/edit_ajax', [UserController::class, 'edit_ajax']);  // Menampilkan halaman form edit user Ajax
    Route::put('/{id}/update_ajax', [UserController::class, 'update_ajax']); // Menyimpan perubahan data user Ajax
    Route::delete('/{id}', [UserController::class, 'destroy']); // Menghapus data user
});
```

4. Then, we create a edit_ajax() function on UserController.php

```
// Menampilkan halaman form edit user ajax
public function edit_ajax(string $id)
{
    $user = UserModel::find($id);
    $level = LevelModel::select('level_id', 'level_nama')->get();

    return view('user.edit_ajax',['user' => $user, 'level' => $level]);
}
```

5. We create **a new view** on the user/edit_ajax.blade.php to display the ajax view form

```
@empty ($user)
    <div id="modal-master" class="modal-dialog-modal-lg" role="document">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title" id="exampleModalLabel">Error</h5>
                <button type="button" class="close" data-dismiss="modal" aria-
label="Close"><span aria-hidden="true">&times;</span></button>
            </div>
            <div class="modal-body">
                <div class="alert alert-danger">
                    <h5><i class="fa-ban fa-ban icon"></i> Error!!</h5>
The data you are looking for is not found</div>
                <a href="{{ url('/user') }}" class="btn btn-warning">Return</a>
```

```
            </div>
        </div>
    </div>
@else
    <form action="{{ url('/user/' . $user->user_id.'/update_ajax') }}" method="POST" id="form-
edit">
    @csrf
    @method('PUT')
    <div id="modal-master" class="modal-dialog-modal-lg" role="document">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title" id="exampleModalLabel">Edit Data User</h5>
                <button type="button" class="close" data-dismiss="modal" aria-
label="Close"><span aria-hidden="true">&times;</span></button>
            </div>
            <div class="modal-body">
                <div class="form-group">
                    <label>User Level</label>
                    <select name="level_id" id="level_id" class="form-control" required>
                        <option value="">- Select Level -</option>
                        @foreach($level as $l)
                            <option {{ ($l->level_id == $user->level_id)? 'selected' : '' }}
value="{{ $l->level_id }}">{{ $l->level_nama }}</option>
                        @endforeach
                    </select>
                    <small id="error-level_id" class="error-text form-text text-
danger"></small>
                </div>
                <div class="form-group">
                    <label>Username</label>
                    <input value="{{ $user->username }}" type="text" name="username"
id="username" class="form-control" required>
                    <small id="error-username" class="error-text form-text text-
danger"></small>
                </div>
                <div class="form-group">
                    <Label>Name</label>
                    <input value="{{ $user->name }}" type="text" name="name" id="name"
class="form-control" required>
                    <small id="error-name" class="error-text form-text text-danger"></small>
                </div>
                <div class="form-group">
                    <label>Password</label>
                    <input value="" type="password" name="password" id="password" class="form-
control">
                    <small class="form-text text-muted">Ignore if you don't want to change the
password</small>
                    <small id="error-password" class="error-text form-text text-
danger"></small>
                </div>
            </div>
            <div class="modal-footer">
                <button type="button" data-dismiss="modal" class="btn btn-
warning">Cancel</button>
                <button type="submit" class="btn btn-primary">Save</button>
            </div>
        </div>
    </div>
    </form>
    <script>
 $(document).ready(function() {
 $("#form-edit").validate({
rules: {
 level_id: {required: true, number: true},
 username: {required: true, minlength: 3, maxlength: 20},
 name: {required: true, minlength: 3, maxlength: 100},
```

```
 password: {minlength: 6, maxlength: 20}
},
 submitHandler: function(form) {
$.ajax({
url: form.action,
type: form.method,
data: $(form).serialize(),
 success: function(response) {
                            if(response.status){
 $('#myModal').modal('hide');
Swal.fire({
 icon: 'success',
 title: 'Succeed',
text: response.message
});
dataUser.ajax.reload();
 }else{
 $('.error-text').text('');
 $.each(response.msgField, function(prefix, val) {
 $('#error-'+prefix).text(val[0]);
});
Swal.fire({
 icon: 'error',
 title: 'Something Went Wrong',
text: response.message
});
}
}
});
                    return false;
},
 errorElement: 'span',
 errorPlacement: function (error, element) {
 error.addClass('invalid-feedback');
 element.closest('.form-group').append(error);
},
 highlight: function (element, errorClass, validClass) {
 $(element).addClass('is-invalid');
},
 unhighlight: function (element, errorClass, validClass) {
 $(element).removeClass('is-invalid');
}
});
});
    </script>
@endempty
```

6. Next, we create a `update_ajax()` function on the `UserController.php` to accommodate user data update requests via ajax

```
public function update_ajax(Request $request, $id){
    Check if the request is from ajax
    if ($request->ajax() || $request->wantsJson()) {
$rules = [
            'level_id' => 'required|integer',
            'username' => 'required|max:20|unique:m_user,username,'.$id.',user_id',
            'name' => 'required|max:100',
            'password' => 'nullable|min:6|max:20'
];

        use Illuminate\Support\Facades\Validator;
$validator = Validator::make($request->all(), $rules);

        if ($validator->fails()) {
            return response()->json([
```

```
                'status' => false,  json response, true: successful, false: failed
                'message' => 'Validation failed.',
                'msgField' => $validator->errors()  indicates which field is the error
]);
}

$check = UserModel::find($id);
        if ($check) {
            if(!$request->filled('password') ){  if the password is not filled, then remove it
from the request
 $request->request->remove('password');
}

$check->update($request->all());
            return response()->json([
                'status' => true,
                'message' => 'Data updated successfully'
]);
 } else{
            return response()->json([
                'status' => false,
                'message' => 'Data not found'
]);
}
}
    return redirect('/');
}
```

7. Now let's try the edit user section, observe the process. Don't forget to report and *commit* to your *git repository*!

Practicum 3. Ajax Modal Delete Data (User Data)

1. In this Practicum 3, we will do the coding for the deletion process using ajax.
2. First of all, let's change the `routes/web.php` to accommodate the confirmation page request to delete the data

```
Route::group(['prefix' => 'user'], function () {
    Route::get('/', [UserController::class, 'index']);          // Menampilkan halaman awal user
    Route::post('/list', [UserController::class, 'list']);      // Menampilkan data user dalam bentuk json untuk datatables
    Route::get('/create', [UserController::class, 'create']);   // Menampilkan halaman form tambah user
    Route::post('/', [UserController::class, 'store']);         // Menyimpan data user baru
    Route::get('/create_ajax', [UserController::class, 'create_ajax']);   // Menampilkan halaman form tambah user Ajax
    Route::post('/ajax', [UserController::class, 'store_ajax']);         // Menyimpan data user baru Ajax
    Route::get('/{id}', [UserController::class, 'show']);       // Menampilkan detail user
    Route::get('/{id}/edit', [UserController::class, 'edit']);  // Menampilkan halaman form edit user
    Route::put('/{id}', [UserController::class, 'update']);     // Menyimpan perubahan data user
    Route::get('/{id}/edit_ajax', [UserController::class, 'edit_ajax']);  // Menampilkan halaman form edit user Ajax
    Route::put('/{id}/update_ajax', [UserController::class, 'update_ajax']); // Menyimpan perubahan data user Ajax
    Route::get('/{id}/delete_ajax', [UserController::class, 'confirm_ajax']); // Untuk tampilkan form confirm delete user Ajax
    Route::delete('/{id}/delete_ajax', [UserController::class, 'delete_ajax']); // Untuk hapus data user Ajax
    Route::delete('/{id}', [UserController::class, 'destroy']); // Menghapus data user
});
```

3. Then we create the `confirm_ajax()` function on `UserController.php`

```
public function confirm_ajax(string $id){
    $user = UserModel::find($id);

    return view('user.confirm_ajax', ['user' => $user]);
}
```

4. Next we view to confirm delete data with `user/confirm_ajax.blade.php name`

```
@empty ($user)
    <div id="modal-master" class="modal-dialog-modal-lg" role="document">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title" id="exampleModalLabel">Error</h5>
                <button type="button" class="close" data-dismiss="modal" aria-
label="Close"><span aria-hidden="true">&times;</span></button>
            </div>
            <div class="modal-body">
                <div class="alert alert-danger">
                    <h5><i class="fa-ban fa-ban icon"></i> Error!!</h5>
The data you are looking for is not found</div>
                <a href="{{ url('/user') }}" class="btn btn-warning">Return</a>
            </div>
        </div>
    </div>
@else
    <form action="{{ url('/user/' . $user->user_id.'/delete_ajax') }}" method="POST" id="form-
delete">
    @csrf
    @method('DELETE')
    <div id="modal-master" class="modal-dialog-modal-lg" role="document">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title" id="exampleModalLabel">Delete User</h5 Data>
                <button type="button" class="close" data-dismiss="modal" aria-
label="Close"><span aria-hidden="true">&times;</span></button>
            </div>
            <div class="modal-body">
                <div class="alert alert-warning">
                    <h5><i class="icon fas fa-ban"></i> Confirm !!</h5>
Do you want to delete data like below?
                </div>
                <table class="table table-sm table-bordered table-striped">
                    <tr><th class="text-right col-3">User Level :</en><td class="col-9">{{
$user->level->level_nama }}</td></tr>
                    <tr><th class="text-right col-3">Username :</th><td class="col-9">{{
$user->username }}</td></tr>
                    <tr><th class="text-right col-3">Name :</th><td class="col-9">{{ $user-
>name }}</td></tr>
                </table>
            </div>
            <div class="modal-footer">
                <button type="button" data-dismiss="modal" class="btn btn-
warning">Cancel</button>
                <button type="submit" class="btn btn-primary">Yes, delete</button>
            </div>
        </div>
    </div>
    </form>
    <script>
 $(document).ready(function() {
 $("#form-delete").validate({
rules: {},
 submitHandler: function(form) {
$.ajax({
url: form.action,
type: form.method,
data: $(form).serialize(),
 success: function(response) {
                            if(response.status){
 $('#myModal').modal('hide');
Swal.fire({
 icon: 'success',
 title: 'Succeed',
```

```
text: response.message
});
dataUser.ajax.reload();
 }else{
 $('.error-text').text('');
 $.each(response.msgField, function(prefix, val) {
 $('#error-'+prefix).text(val[0]);
});
Swal.fire({
 icon: 'error',
 title: 'Something Went Wrong',
text: response.message
});
}
}
});
                    return false;
},
 errorElement: 'span',
 errorPlacement: function (error, element) {
 error.addClass('invalid-feedback');
 element.closest('.form-group').append(error);
},
 highlight: function (element, errorClass, validClass) {
 $(element).addClass('is-invalid');
},
 unhighlight: function (element, errorClass, validClass) {
 $(element).removeClass('is-invalid');
}
});
});
    </script>
@endempty
```

5. Then we create a `delete_ajax()` function on `the UserController.php` to accommodate *the* request to delete user data

```php
public function delete_ajax(Request $request, $id)
{
    // cek apakah request dari ajax
    if ($request->ajax() || $request->wantsJson()) {
        $user = UserModel::find($id);
        if ($user) {
            $user->delete();
            return response()->json([
                'status'  => true,
                'message' => 'Data berhasil dihapus'
            ]);
        } else {
            return response()->json([
                'status'  => false,
                'message' => 'Data tidak ditemukan'
            ]);
        }
    }
    return redirect('/');
}
```

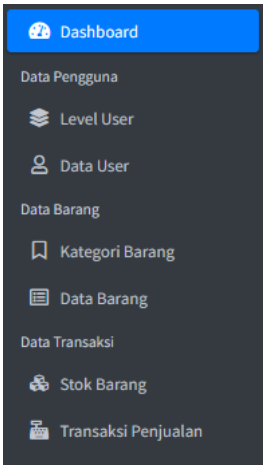
6. Once all is done, let's try to do an experiment of the coding we've done.
7. Don't forget to report to the jobsheet report and commit to your *git repository*.!

F. Jobsheet Tasks

Implement coding for Ajax Form and Client Validation with jQuery Validation in the following menu

- ✓ Table m_level
- ✓ Table m_kategori
- ✓ Table m_supplier
- ✓ Table m_barang

Report it on the jobsheet report and don't forget to commit and push it to your git repository.



*That's all, and happy learning ****